BEACONHOUSE NATIONAL UNIVERSITY

Assignment 01

Name: Saad Mughal

BNU ID: F2023-009

Total Marks: 10
Due Date: 26-10-2025

OS(SEC_C)

Course Instructor	Miss Noreen Khalid
Lab Instructor	Sir Ali Hafeez
Semester	Fall 2025

23 Oct 2025 Assignment - 01 Thursday

Operating Systems

PART A: Introduction to Operating Systems

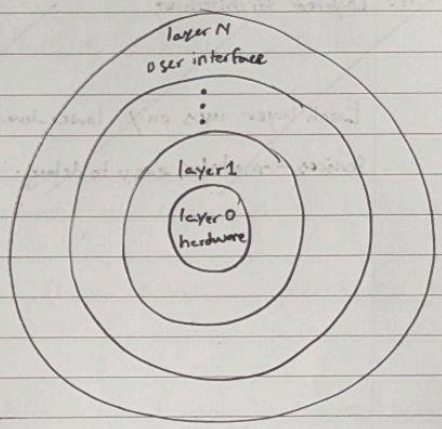
1. Time-Sharing OS as it supports many interactive users/processes concurrently (coding, compiling, design tools) with fair CPU scheduling and fast context switches. It also maximizes resource sharing (printers, network storage) and keeps the systems responsive for students.

Three essential OS services:

1. Process & CPU Scheduling (create, run, preempt processes/threads).
2. Memory Management & Virtual Memory (isolation, protection, efficient RAM use for IDEs, compilers, design apps).
3. File & I/O Management (permissions, directories, device drivers for printers, GPUs, tablets).

2.

User Applications	6
System Programs	5
I/O Management	4
Memory Management	3
Process Management	2
Hardware	1



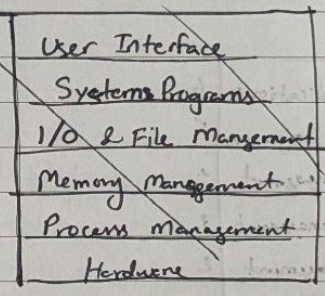
Each layer uses only lower-level services -> modular, easy to debug.

PART B: Operating System Structures

3. - Monolithic: All core services and drivers run in kernel space; fast, but a bad driver can crash the whole OS.
   - Modular (Monolithic with loadable Modules): kernel can load/unload drivers at runtime; still kernel-space, so a faulty module can still panic the kernel, but easier maintenance and updates.
   - Microkernel: Only minimal core (IPC, scheduling, basic memory) in kernel; drivers & services in user space communicating via message passing → strong isolation; a failing driver doesn't crash the whole system (it can be restarted).

   Recomendation: Microkernel (or microkernel-style separation) to reduce crashes, driver faults are isolated in user space and can be restarted without a full reboot.

4. Layered Architecture



Each layer uses only lower-level services → modular, easy to debug.

8. → An interrupt is an asynchronous signal that pauses the current CPU ~~event~~ execution so the OS can run an interrupt Service Routine (ISR).

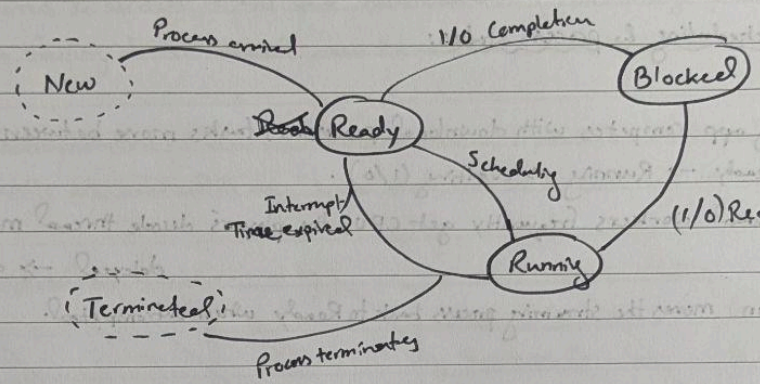
→ Enables fast I/O handling, timer ticks for scheduling, and transitions of processes (e.g. waking a waiting process when I/O completes).

Examples:

(•) Hardware interrupt: keyboard keypress or timer tick.

(•) Software interrupt (trap): Divide-by-zero, or a system call (e.g., int 0x80/svc).

9.



- New → Ready : Admitted into RAM
- Ready → Running : Scheduled by CPU
- Running → Watching : Blocked fo I/O
- Waiting → Ready : Event complete
- Running → Terminate: Process finished.

5. Best structure: Hybrid (Modular + Microkernel concepts).

-> Performance-critical paths (graphics, multimedia) can remain in kernel / fast paths, while sensor/services live as replaceable modules -> good performance + frequent OTA updates.

-> Better fault isolation/security than pure monolithic: many services/daemons are sandboxed; vendor drivers can be updated without full OS rewrite.

PART C: Process Management & Interrupts

6. CPU scheduling & process states:

- Streaming app competes with download processes; tasks move between Ready -> Running -> Waiting (I/O).

If download workers frequently get CPU, the player's decode thread may be delayed -> stutter/lags.

- Preemption moves the streaming process back to Ready when outcompeted.

Priority scheduling to improve UX:

- Give the player higher priority so its decode/audio threads preempt download tasks.
- Optionally use I/O and network QoS (lower I/O/network priority for downloads) so buffering stays smooth.

7. Why synchronization matters: Account updates are critical sections, ~~must be~~ Must be atomic to prevent inconsistent balances when operations overlap.

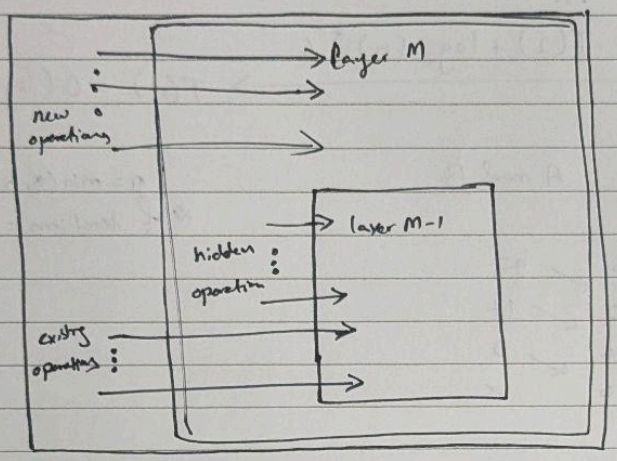
If sync fails:-

- Race conditions/Lost updates: Two withdrawals of $100 on $150 run concurrently. Each reads $150, both write back $50 -> One withdrawal effectively ignored -> balance ~~was~~ Wrong.
- Inconsistency/Overdrafts, deadlocks (if locks misused), dirty/partial writes. ~~(Use locks~~ (Use locks/semaphores/transactions with ACID properties or DB-level isolation.

10. Hard ~~Real~~ Real-Time OS (deterministic, bounded latencies).
~~Scheduling, typically Earliest Deadline~~
Scheduling algorithm: Preemptive priority real-time scheduling, typically Earliest Deadline First (EDF) for dynamic deadlines or Rate Monotonic Scheduling (RMS) for periodic tasks.

→ Deterministic response to meet strict deadlines; preemption ensures critical alert handlers run immediately; schedulability analysis possible (EDF/RMS) to prove deadlines are met.

4.



Direction	~~Meaning~~	Meaning
Upward (Layer M -> Higher Layers)		Provides new operations or services
Downward (Layer M -> Layer M-1)		Uses existing ~~operation~~ operations provided by the lower layer.
Hidden Operations		Internal helper routines, invisible to higher layer